VERIQTA

The Beginner's Troubleshooting Playbook:

How to Diagnose Real DevOps Failures

Linux
- Permission denied
- Service not starting
- Disk full
- High CPU/Memory

Docker
- Container exited
- Port not accessible
- Image pull errors
- Container logs

Kubernetes
- CrashLoopBackOff
- ImagePullBackOff
- Pod Pending
- Service not reachable

Network
- Connection refused
- DNS not resolved
- Timeout
- Firewall blocking



CI/CD
- Build failed
- Tests failed
- Deployment failed
- Pipeline errors

Git
- Merge conflict
- Wrong branch
- Push rejected
- Lost changes

Applications
- HTTP 500/502/503
- Service unavailable
- Configuration errors
- App not responding

Cloud
- Instance unreachable
- Security group issue
- Service unavailable
- Access denied

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

PAGE 01

The First 60 Seconds of a Failure

VERIQTA

A deployment has completed, but users report that the application is unavailable. What should you check before changing anything?

Learning Goals

- Understand what to do in the first 60 seconds.
- Establish the symptoms and who is affected.
- Check recent changes and collect evidence.
- Learn the difference between a symptom and a root cause.

Key Takeaway

Do not change anything until you understand what is failing, who is affected, and why it might be happening.

The Troubleshooting Workflow

1. OBSERVE

- What is failing?
- What are the symptoms?
- Who is affected?
- What errors are seen?

2. HYPOTHESIZE

- What might be causing this?
- List possible reasons.
- Use evidence, not guesswork.

3. TEST

- Run diagnostic commands.
- Check logs.
- Verify your assumption.
- Gather more evidence.

4. FIX

- Apply the correct fix.
- Make the smallest safe change.
- Avoid random changes.

5. VERIFY

- Confirm the issue is resolved.
- Check that users can access the application.
- Monitor for recurrence.

Symptom vs Root Cause

Symptom

What you can see or experience. It shows that something is wrong but not the real reason it happened.

Example

- Website shows 502 error
- Application is slow
- Container keeps restarting

Root Cause

The actual underlying reason why the problem is happening. Fixing the root cause prevents the issue from coming back.

Example

- Misconfigured upstream
- Database connection limit
- Application crash due to code

First 60 Seconds Checklist

1. Confirm the reported issue.
2. Identify which users or services are affected.
3. Check recent changes (deployments, configurations, infrastructure).
4. Collect initial evidence (errors, logs, metrics).
5. Decide the next step based on evidence.

Beginner Mistake

Making changes without understanding the problem. This can make the issue worse and hide the real cause.

Practice This

The next time you see a failure, write down the 5 steps (Observe → Hypothesize → Test → Fix → Verify) before you make any changes.

PAGE 02

Read the Error Before You Fix It

VERIQTA

A command fails with Permission denied, command not found, or No such file or directory.
Before searching online or copying a fix, read and understand the error.

Learning Goals

- Identify the failed operation.
- Read the exact error message.
- Distinguish different failure categories.
- Avoid copying random fixes.
- Use basic commands to diagnose issues.

Key Takeaway

The error message tells you what failed and often why it failed.

Understanding the error helps you choose the right fix instead of guessing.

Common Errors and How to Diagnose Them

1. Permission Denied

```
$ cat /etc/shadow
cat: /etc/shadow:
Permission denied
```

What it means:
You do not have the required permissions to access the file or run the command.

How to diagnose:

- Check your current user: `id`
- Check file permissions: `ls -l`
- Check if sudo is required.

2. Command Not Found

```
$ nginx -v
bash: nginx: command
not found
```

What it means:
The command or program is not installed or not in your PATH.

How to diagnose:

- Check if it exists: `command -v nginx`
- Check your PATH: `echo $PATH`
- Install the package if needed.

3. No Such File or Directory

```
$ cd /var/www/html2
bash: cd: /var/www/html2:
No such file or directory
```

What it means:
The file or directory path you specified does not exist or is incorrect.

How to diagnose:

- Check the correct path: `pwd`
- List the parent directory: `ls -l`
- Look for typos in the name.

Key Commands for Investigation

Command	What It Does
`pwd`	Shows your current directory.
`ls -l`	Lists files and shows permissions.
`command -v <name>`	Checks if a command exists.
`id`	Shows your current user and groups.

Practical Examples

Example 1

```
$ pwd
/home/user
```

Use this to confirm your current directory before running a command.

Example 2

```
$ ls -l /etc/nginx
ls: cannot access
'/etc/nginx': No such file or directory
```

Check if the path exists and is correct.

Example 3

```
$ command -v docker
docker
```

Shows the full path if the command is installed. If nothing is returned, it is not installed.

Beginner Mistake

Copying random commands from the internet without understanding the error or what the command changes.

Practice This

Intentionally run the three errors on your system.
Use `pwd`, `ls -l`, `command -v`, and `id` to identify the cause and write down the correct fix.

PAGE 03

Find the Evidence Hidden in Logs

VERIQTA

An application stops responding, but its logs contain hundreds of messages.
Your job is to find the real cause, not just read every line.

Learning Goals

- Understand log structure and timestamps.
- Identify severity levels (info, warning, error).
- Find the real error in noisy logs.
- Use journalctl, tail, grep, and less effectively.
- Understand the difference between a warning and the actual failure.

Key Takeaway

Logs tell a story. Look for:

- When the problem started (timestamps).
- What type of message it is (severity level).
- The context around the error (the lines before and after).
- The real cause, not just warnings.

Understanding Log Messages

```
Oct 18 10:23:11  myapp[1234]:  INFO   Starting server
Oct 18 10:23:12  myapp[1234]:  WARN   Using default config
Oct 18 10:23:15  myapp[1234]:  ERROR  Failed to connect to DB
Oct 18 10:23:15  myapp[1234]:  ERROR  Exiting application
```

Timestamp	Service/Process	Severity Level	Message
Tells you when the event happened.	Shows which application logged it.	Common levels: INFO, WARN, ERROR	The actual event and details.

Common Log Severity Levels

DEBUG	Detailed information (useful for troubleshooting).
INFO	Normal operational messages.
WARN	Something unexpected but not a failure.
ERROR	A clear problem that needs attention.
CRITICAL	A serious failure, often causing service downtime.

Key Commands for Investigating Logs

1. journalctl

```
journalctl -u nginx
```

- Shows logs for a service (e.g., nginx).
- Useful for systemd services.
- Add -f to follow live logs.

Example:

```
journalctl -u nginx -f
```

2. tail

```
tail -n 50 /var/log/nginx/error.log
```

- Shows the last lines of a log file.
- Use -n to specify number of lines.
- Use -f to follow new logs.

Example:

```
tail -f /var/log/nginx/error.log
```

3. grep

```
grep -i error /var/log/nginx/error.log
```

- Searches for specific text (e.g., error).
- Use -i to ignore case.
- Helps you find relevant lines quickly.

Example:

```
grep -i failed /var/log/nginx/error.log
```

4. less

```
less /var/log/nginx/error.log
```

- Opens a log file for easy reading.
- Use arrow keys to navigate.
- Press q to quit.

Example:

```
less +F /var/log/nginx/error.log
```

Warning vs Actual Failure

Warning (Not always a problem)

```
WARN  Using default config
```

This may not cause a failure. Keep looking for errors.

Actual Failure

```
ERROR  Failed to connect to DB
```

This is likely the real cause. Check the lines before and after for more context.

Practice This

1. View the last 50 lines of a real service log using tail.
2. Search for error messages using grep.
3. Open the log file with less and read the lines before and after the error.
4. Identify the first error that explains the failure.

PAGE 04

Permission Denied: Who Can Access This File?

VERIQTA

A deployed application cannot read its configuration file.
Learn how Linux file ownership, permission bits, service users, and directory traversal permissions work. See why using **chmod 777** is a dangerous shortcut.

Learning Goals

- Understand Linux file ownership and permission bits.
- Learn how service users access files.
- Understand directory traversal permissions.
- Diagnose permission issues using commands.
- Choose the least-privileged fix (not chmod 777).

Key Takeaway

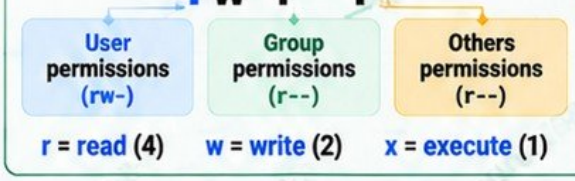
- A file can be readable, but you also need permission to access all parent directories.
- Use the correct ownership and permissions for the application user.
- chmod 777 gives everyone full access and is a security risk.

Example: Permission Denied Error

```
$ cat /etc/myapp/config.yaml
cat: /etc/myapp/config.yaml:
Permission denied
```

This means:
Your user does not have the required permissions to read this file.

Understanding Permission Bits



Check File Ownership and Permissions

```
$ ls -l /etc/myapp/config.yaml
-rw------- 1 root myapp 1234 Oct 18 10:21
/etc/myapp/config.yaml
```

- This shows the file permissions.
- The file is owned by user `root` and group `myapp`.
- Only root can read and write it.

Check Directory Traversal Permissions

```
$ namei -l /etc/myapp/config.yaml
f:  /etc/myapp/config.yaml
  drwxr-xr-x   /
  drwxr-xr-x   etc
  drwxr-x---   myapp
  -rw-------   config.yaml
```

Every directory in the path must have **execute (x)** permission for the user to traverse it.

Check the Application User

```
$ id  myapp
uid=1001(myapp) gid=1001(myapp)
groups=1001(myapp)
```

This shows the user and group that the application runs as. Use this user when setting permissions.

Fix the Problem (Least-Privileged Approach)

```
$ sudo chown myapp:myapp /etc/myapp/config.yaml
$ sudo chmod 640 /etc/myapp/config.yaml
```

- Give ownership to the application user or group.
- Allow only the required permissions (e.g., 640).
- Avoid chmod 777 unless there is a very specific reason, and you understand the risk.

Why chmod 777 Is Dangerous

- Gives full access to everyone.
- Any user on the server can read, modify, or delete the file.
- Can expose sensitive information (e.g., passwords, API keys).
- Violates the principle of least privilege.

Practice This

1. Use `ls -l` to check the permissions of a file.
2. Use `namei -l` to check directory traversal permissions.
3. Use `id` to check the application user.
4. Fix the permissions using the least-privileged approach.

PAGE 05

The Service Is Down: What Failed?

VERIQTA

Nginx fails to start after an engineer changes its configuration.
Learn how to check the service status, read logs, test the configuration, and identify the real error before restarting.

Learning Goals

- Understand what an installed service, enabled service, and running service mean.
- Learn how to check the status of a service using systemctl.
- Find configuration errors using nginx -t.
- Read detailed logs with journalctl.
- Identify the real cause before restarting the service.

Key Takeaway

- A service can be installed, enabled, but not running.
- Always check the configuration before restarting.
- Logs show the real reason a service fails.
- Fix the actual error, not just the symptom.

The Three Service States

Installed

- The service files are present on the system.
- You can see it with systemctl list-unit-files.
- It does not mean the service is running.

Enabled

- The service is configured to start on boot.
- You can see it with systemctl is-enabled.
- It does not mean the service is currently running.

Running

- The service is actively running right now.
- You can see it with systemctl status.
- This is what you want when the application is working.

Check Service Status

```
$ systemctl status nginx
● nginx.service - A high performance web server
   Loaded: loaded (/usr/lib/systemd/...)
   Active: failed (Result: exit-code)
     Docs: man:nginx(8)
```

What to look for:

- Is the service running or failed?
- Read the error message in the status output.
- Check the Active line and the recent log lines.

Test the Nginx Configuration

```
$ nginx -t
nginx: [emerg] invalid number of arguments in "/etc/nginx/nginx.conf:10"
nginx: configuration file /etc/nginx/nginx.conf test failed
```

What this does:

- Checks the syntax of your Nginx configuration files.
- Shows the exact file and line number if there is an error.
- Fix the error before trying to start or restart the service.

Read Detailed Logs

```
$ journalctl -u nginx -n 50 --no-pager
Oct 18 10:21:34 server nginx[1234]: nginx: [emerg] invalid number of arguments in "/etc/nginx/nginx.conf:10"
Oct 18 10:21:34 server systemd[1]: nginx.service: control process exited, code=exited, status=1/FAILURE
Oct 18 10:21:34 server systemd[1]: nginx.service failed.
```

What to look for:

- The actual error message (not just "service failed").
- The file and line number causing the problem.
- Events around the time the service was started.

Common Causes of Nginx Failure

- Syntax error in the configuration file.
- Wrong file path or missing include file.
- Port already in use.
- Permission issues on configuration files or directories.
- Invalid directive or wrong parameters.
- Missing dependencies (e.g., certificates, modules).

Practice This

1. Run systemctl status nginx and check the output.
2. Run nginx -t and identify the configuration error.
3. Use journalctl -u nginx to read the detailed logs.
4. Fix the configuration file and test again with nginx -t.
5. Start the service and confirm it is running.

PAGE 06

Disk Full: Why Applications Suddenly Fail

VERIQTA

A running application can no longer write logs or temporary files.

Learn about filesystem capacity, inode exhaustion, log growth, and the difference between disk usage and available memory.

Learning Goals

- Understand why a "disk full" error happens.
- Learn the difference between disk space and inodes.
- Identify log growth and other common causes.
- Use df -h, df -i, and du -sh to locate the actual problem.
- Learn safe troubleshooting steps without deleting unknown files.

Key Takeaway

- A system can run out of disk space or inodes.
- Both can cause applications to fail.
- Disk usage shows how much space is used.
- Inodes show how many files can be created.
- High log growth is a common cause.
- Always identify the real cause before taking action.

1. Check Disk Space Usage (df -h)

```
$ df -h
Filesystem   Size  Used  Avail  Use% Mounted on
/dev/sda1    50G   49G   500M   99%  /
tmpfs        1.9G  100M  1.8G    6%  /run
/dev/sdb1    100G   20G   80G   21%  /data
```

- Shows disk space usage for each filesystem.
- Use% close to 100% means the disk is full.
- Avail shows the remaining space.

2. Check Inode Usage (df -i)

```
$ df -i
Filesystem   Inodes   IUsed    IFree   IUse% Mounted on
/dev/sda1    3276800  3276600    200   100%  /
tmpfs         486604      320  486284    1%  /run
/dev/sdb1    6553600   120000 6433600    2%  /data
```

- Shows inode (file) usage.
- If IUse% is 100%, you cannot create new files even if there is still free disk space.
- This often causes applications to fail.

3. Find Large Directories (du -sh)

```
$ sudo du -sh /var/* | sort -h
12M     /var/cache
2.5G    /var/log
50G     /var/lib
100G    /var/www
```

- Shows the size of directories.
- Helps you find where the disk space is being used.
- Use sort -h to see the largest folders first.

4. Check Log Growth

```
$ ls -lh /var/log/
$ sudo du -sh /var/log/* | sort -h
```

- Log files can grow very large and fill the disk.
- Check file sizes and find the largest log files.
- Use log rotation to prevent future issues.

5. Disk Usage vs Memory

Resource	What It Shows
Disk Usage (df -h)	How much storage space is used on disk.
Inodes (df -i)	How many files can be created.
Memory (free -h)	Available RAM in the system.
They Are Different	You can have free memory but still run out of disk space or inodes.

Common Signs of a Disk Full Issue

- Applications cannot write logs or temporary files.
- Errors like "No space left on device".
- Services fail to start.
- Unexpected application crashes.
- System becomes slow or unresponsive.

Safe Steps to Take

1. Identify whether the issue is disk space or inodes.
2. Find the largest directories and files.
3. Check log files and rotate or archive them.
4. Remove only files you understand.
5. Consider increasing disk size or enabling log rotation.

Practice This

1. Run df -h and identify any filesystems that are full.
2. Run df -i and check inode usage.
3. Use du -sh to find the largest directory in /var.
4. Identify the top 3 largest folders.
5. Write a short summary of what is causing the problem.

PAGE 07

High CPU, Low Memory, or a Killed Process?

VERIQTA

An application becomes slow and occasionally disappears.
Learn about CPU utilization, memory pressure, the Linux Out-of-Memory killer, and the difference between a busy process and a failing process.

Learning Goals

- Understand CPU utilization and what high CPU means.
- Understand memory usage and memory pressure.
- Learn how the Linux Out-of-Memory (OOM) killer works.
- Distinguish between a busy process and a failing process.
- Use top, free -h, ps, and kernel logs to investigate.
- Decide whether the issue needs a code fix, resource adjustment, or further investigation.

Key Takeaway

- High CPU means the system is busy, but it does not always mean a failure.
- Low memory or high memory usage can cause the OOM killer to terminate processes.
- A process can be slow due to CPU, memory, disk I/O, or external dependencies.
- Always check evidence (metrics and logs) before making changes.

1. Check CPU Usage (top)

```
$ top
%Cpu(s): 85.6 us, 10.2 sy,  0.0 ni,  4.0 id,
0.2 wa,  0.0 hi,  0.0 si,  0.0 st
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	COMMAND
1234	app	20	0	1620m	120m	1234	R	92.5	3.1	myapp
567	root	20	0	324m	45m	2345	S	12.3	1.2	nginx
890	app	20	0	512m	80m	3210	S	5.6	2.1	worker

- %CPU shows how much CPU a process is using.
- R means the process is running and using CPU.
- High CPU for a long time can cause the application to be slow.

2. Check Memory Usage (free -h)

```
$ free -h
              total        used        free      shared  bu
Mem:          7.7G        7.4G        150M        320M  1.2
Swap:         2.0G        1.1G        900M
```

- Shows total, used, and available memory.
- Low available memory can cause the system to become slow.
- If memory is fully used, the OOM killer may terminate processes.

3. Check Running Processes (ps)

```
$ ps aux --sort=-%mem | head
USER       PID  %CPU  %MEM  COMMAND
app       1234  10.4  12.5  myapp
root       567   1.2   5.6  java
app        890   0.8   4.3  worker
```

- Lists running processes with CPU and memory usage.
- Helps identify which process is using the most resources.

4. Check for OOM Killer in Kernel Logs

```
$ journalctl -k | grep -i oom
Oct 18 10:23:11 server kernel: Out of memory: Killed
process 1234 (myapp) total-vm:1620000kB, anon-rss:512000kB
Oct 18 10:23:11 server kernel: oom_reaper: reaped process
1234 (myapp), now anon-rss:0kB
```

- The kernel log shows if the OOM killer terminated a process.
- Look for messages like "Out of memory" and "Killed process."
- This confirms that memory pressure caused the process to exit.

5. Understand the Difference

Situation	What It Means	Example	What to Do
High CPU	Process is using a lot of CPU.	%CPU > 80% in top	Check the process, code, or traffic.
Low Memory	System is running out of memory.	free -h shows little available memory.	Find memory heavy process or increase resources.
OOM Killer	Kernel terminates a process due to memory pressure.	"Out of memory: Killed process" in logs.	Check why it used so much memory and fix it.
Busy Process	Process is working hard but not failing.	High CPU but still running normally.	Check if this is expected (e.g., high traffic, batch job).
Failing Process	Process crashes or is killed.	Process disappears or restarts.	Check logs, fix the root cause, adjust resources.

Common Causes

- Memory leak in the application.
- Traffic spike or heavy workload.
- Inefficient code or database queries.
- Large files or logs consuming memory.
- Insufficient system resources.

How to Decide What to Do

1. Check CPU and memory usage.
2. Identify the process causing the issue.
3. Look for OOM killer messages in logs.
4. Decide if the issue is code related, resource related, or external.
5. Take the appropriate action (fix, scale, or investigate further).

Practice This

1. Run top and identify the top CPU consuming process.
2. Run free -h and check available memory.
3. Run ps aux --sort=-%mem and find the top memory consuming process.
4. Check kernel logs for OOM killer messages.
5. Write a short summary of what is causing the issue and what action you would take.

PAGE 08

Connection Refused: Is Anything Listening?

VERIQTA

An application attempts to reach a backend on port 8080 but receives a connection-refused error.

Learn about listening sockets, ports, loopback addresses, service binding, and the difference between a refused connection and a timeout.

Learning Goals

- Understand what a listening socket is.
- Learn how ports and addresses work (localhost vs 0.0.0.0).
- Understand service binding and why a service may not be reachable.
- Differentiate between a refused connection and a timeout.
- Use ss -lntp and curl -v to investigate the issue.

Key Takeaway

- A connection refused error means no service is listening on the target address and port.
- A timeout means the request did not get a response (firewall, network issue, or service not reachable).
- A service can be running but only listening on 127.0.0.1, making it inaccessible from other machines.
- Always check what address and port the service is actually listening on.

1. Check Listening Ports (ss -lntp)

```
$ ss -lntp | grep 8080
State    Recv-Q  Send-Q   Local Address:Port   Peer Address
LISTEN   0       128      0.0.0.0:8080         0.0.0.0:*
# or
$ sudo ss -lntp | grep 8080
```

- Shows which processes are listening on which ports.
- Check the Local Address to see if it is 127.0.0.1 (localhost) or 0.0.0.0 (all interfaces).
- If nothing is shown, no service is listening on that port.

2. Test the Connection (curl -v)

```
$ curl -v http://localhost:8080
*   Trying 127.0.0.1:8080...
* connect to 127.0.0.1 port 8080 failed:
  Connection refused
```

```
$ curl -v http://192.168.1.10:8080
*   Trying 192.168.1.10:8080...
* Operation timed out after 5000 milliseconds
```

- Connection refused = no service is listening.
- Operation timed out = request did not get a response (network, firewall, or service not reachable).
- Use curl -v to see detailed connection information.

3. Understand Addresses and Binding

Address	What It Means
127.0.0.1 (localhost)	Only accessible from the same machine.
0.0.0.0	Listens on all network interfaces (accessible from other machines).
192.168.1.10	Listens on a specific network interface.

If a service is running but only bound to 127.0.0.1, other machines cannot connect to it.

4. Common Causes of Connection Refused

- The application or service is not running.
- The service is running but listening on the wrong port.
- The service is only listening on 127.0.0.1.
- A firewall is blocking the port.
- The service crashed or failed to start.
- A configuration change caused the service to stop listening.

5. Connection Refused vs Timeout

Error	Meaning
Connection refused	No service is listening on the target address and port.
Operation timed out	The request did not get a response (network issue, firewall, or service unreachable).

Practice This

1. Use ss -lntp to check if anything is listening on port 8080.
2. Use curl -v http://localhost:8080 and observe the result.
3. Try curl -v http://<server-ip>:8080 and compare the result.
4. Identify whether the service is not running, bound to the wrong address, or blocked by a firewall.
5. Summarize the issue and decide the next step to fix it.

PAGE 09

DNS Failure: The Name Cannot Be Resolved

VERIQTA

A server can reach an IP address but cannot connect using the application's hostname.

Learn about DNS resolution, DNS records, resolvers, and why a DNS problem differs from a service or routing problem.

Learning Goals

- Understand how DNS resolution works.
- Learn about common DNS records (A, AAAA, CNAME, etc).
- Understand the role of resolvers (e.g., /etc/resolv.conf).
- Learn how to diagnose DNS failures using getent, dig, and curl.
- Differentiate a DNS problem from a service or routing problem.

Key Takeaway

- DNS translates a domain name to an IP address.
- If DNS fails, the hostname cannot be resolved, even if the service is running.
- A DNS problem is different from a service problem.
- A service problem occurs after the name is resolved, but the connection fails.
- Use multiple tools to confirm whether the issue is name resolution, network connectivity, or the service itself.

1. Check Name Resolution with getent

```
$ getent hosts example.com
93.184.216.34   example.com
```

- Shows the IP address returned by the system's DNS resolver.
- If no output is returned, the hostname cannot be resolved.

2. Check DNS Records with dig

```
$ dig example.com
;; ANSWER SECTION:
example.com.   300  IN   A
93.184.216.34
```

- Queries DNS directly and shows detailed information.
- Helps you check A, AAAA, CNAME and other DNS records.
- Useful for troubleshooting DNS configuration issues.

3. Test the Connection with curl

```
$ curl -v http://example.com
*   Trying 93.184.216.34:80...
*  Connected to example.com
   (93.184.216.34) port 80
>  GET / HTTP/1.1
>  Host: example.com
```

- Shows the full connection process.
- If the hostname cannot be resolved, you will see an error like: "Could not resolve host".

Common DNS Commands

Command	Purpose
getent hosts <domain>	Check name resolution using system resolver.
dig <domain>	Query DNS and show detailed records.
curl -v <url>	Test name resolution and connection.
cat /etc/resolv.conf	View the DNS servers configured on the system.

Common DNS Records

Record	Purpose
A	Maps a domain name to an IPv4 address.
AAAA	Maps a domain name to an IPv6 address.
CNAME	Alias to another domain name.
MX	Mail server record.
NS	Authoritative name server.
TXT	Additional information (e.g., SPF, verification).

Where DNS Settings Are Stored

File / Tool	Purpose
/etc/resolv.conf	DNS servers used by the system.
/etc/hosts	Local hostname mappings (checked before DNS).
systemd-resolved	DNS resolver service (on many Linux systems).

Signs of a DNS Problem

- Cannot resolve hostname error.
- getent returns no results.
- dig fails to return a record.
- Applications cannot connect using a domain name but can connect using the IP address.
- Other services on the server can resolve names.

DNS vs Service vs Routing Problem

Problem Type	What It Means
DNS Problem	Hostname cannot be resolved to an IP address.
Service Problem	Hostname resolves, but the service is not running or not listening on the port.
Routing Problem	Hostname resolves, but the network path to the IP address is blocked or unreachable.

Practice This

1. Use getent hosts <domain> and check the result.
2. Use dig <domain> and review the DNS records.
3. Use curl -v http://<domain> and observe the error or success.
4. Compare the results and identify whether the issue is DNS, service, or routing.
5. Suggest the next steps to fix the problem.

VERIQTA

PAGE 10

HTTP 500, 502, 503, or 504: Where Did It Break?

VERIQTA

Users receive an error from a website, but the frontend and backend report different conditions.

Learn how a reverse proxy communicates with an application and why each status code provides different clues.

Learning Goals

- Understand how a reverse proxy (Nginx) communicates with an application.
- Know what HTTP 500, 502, 503, and 504 mean.
- Identify where the failure occurred (client, proxy, or application).
- Use curl -i, proxy logs, and application logs to trace requests.
- Learn a structured approach to troubleshoot web application errors.

Key Takeaway

- A reverse proxy (Nginx) sits between the client and the application.
- Each HTTP error code gives a different clue about where the failure happened.
- 500 = Application error (the app returned an error).
- 502 = Bad gateway (proxy got an invalid response from the app).
- 503 = Service unavailable (app is down or overloaded).
- 504 = Gateway timeout (proxy did not get a response in time).
- Check logs on both Nginx and the application server to find the root cause.

1. How the Request Flows



- The client sends a request to Nginx.
- Nginx forwards the request to the application.
- The application processes the request and returns a response.
- Nginx sends the response back to the client.
- If something fails, the error code tells you where it likely broke.

2. Common HTTP Error Codes

Status Code	Meaning	What It Usually Indicates
500 Internal Server Error	The application returned an error.	Bug in application code, uncaught exception, or misconfiguration.
502 Bad Gateway	Nginx received an invalid response from the application.	Application is down, crashed, or returned an invalid response.
503 Service Unavailable	The application is not available.	Application is not running, overloaded, or under maintenance.
504 Gateway Timeout	Nginx did not get a response in time from the application.	Application is slow or not responding.

3. Test the Response with curl

```
$ curl -i http://example.com
HTTP/1.1 502 Bad Gateway
Server: nginx/1.24.0
Date: Wed, 21 Oct 2026 10:00:00 GMT
Content-Type: text/html
...
```

- The -i option shows the HTTP status code and headers.
- Check the exact error code (500, 502, 503, or 504).
- This tells you what the client is seeing.

4. Check Nginx (Proxy) Logs

```
$ sudo tail -n 20 /var/log/nginx/error.log
2026/10/21 10:00:00 [error] 1234#1234:
upstream timed out (110: Connection
timed out) while reading response header
from upstream, client: 192.168.1.10,
server: example.com, request:
"GET / HTTP/1.1" ...
```

- Nginx logs show if it could connect to the application.
- Look for errors like "upstream timed out", "connect() failed", or "invalid response".
- The logs help you understand whether the problem is at the proxy or application.

5. Check Application Logs

```
$ sudo journalctl -u myapp -n 20
Oct 21 10:00:00 server myapp[5678]:
ERROR: Database connection failed
Oct 21 10:00:01 server myapp[5678]:
ERROR: Uncaught exception in request
handler ...
```

- Application logs show the actual error inside the application.
- Look for stack traces, database errors, or uncaught exceptions.
- If the application is not running, fix the issue and restart it.

Practice This

1. Use curl -i http://your-domain.com and note the HTTP status code.
2. Check /var/log/nginx/error.log and access.log for related errors.
3. Check the application logs (e.g. journalctl -u yourapp).
4. Identify whether the error is 500, 502, 503, or 504.
5. Explain where the failure likely occurred and what to check next.

Troubleshooting Steps

1. Check the HTTP status code with curl -i.
2. Review Nginx error and access logs.
3. Check if the application is running and healthy.
4. Look at application logs for the real error.
5. Fix the issue and test again.

Key Questions to Ask

1. Is Nginx running and reachable?
2. Is the application running?
3. Is Nginx able to connect to the application?
4. What error does the application log show?
5. Is the application slow or crashing?

PAGE 11

Git Says Conflict: What Actually Happened?

VERIQTA

Two engineers change the same lines, and Git cannot automatically combine their work.

Learn about conflicting changes, conflict markers, and why resolving a conflict means understanding the intended final code.

Learning Goals

- Understand why merge conflicts happen.
- Learn how Git detects conflicting changes.
- Identify and read conflict markers.
- Resolve a conflict by choosing or combining the correct changes.
- Use git status, git diff, and commit to complete the resolution.

Key Takeaway

- A conflict happens when two or more people change the same lines in a file.
- Git cannot know which change to keep.
- Conflict markers show the different versions of the code.
- Resolving a conflict means understanding the code and creating the correct final version.
- Always review the changes, test if needed, and commit after resolving.

1. Check the Conflict with git status

```
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort)

Unmerged paths:
   both modified:   app.py
```

- Shows which files have conflicts.
- Files marked as both modified need to be resolved before committing.

2. Open the File and Read Conflict Markers

```
# app.py
1   def get_message():
2   <<<<<< HEAD
3       return "Hello from main"
4   =======
5       return "Hello from feature"
6   >>>>>> feature
```

- `<<<<<<<` shows the current branch (HEAD).
- `=======` separates the changes.
- `>>>>>>>` shows the other branch.
- Both changes cannot remain. You must decide the correct final code.

3. Resolve the Conflict

```
# app.py (resolved)
1   def get_message():
2       return "Hello from main and feature"
```

- Choose one change or combine both changes (if it makes sense).
- Remove the conflict markers (`<<<<<<<`, `=======`, `>>>>>>>`).
- Make sure the final code is correct and works as intended.
- Save the file.

4. Review the Changes with git diff

```
$ git diff
diff --git a/app.py b/app.py
index e69de29..4b825dc 100644
--- a/app.py
++- a/app.py
@@ -1,4 +1,2 @@
-     return "Hello from main"
-     return "Hello from feature"
+     return "Hello from main and feature"
```

- Shows exactly what changed.
- Verify the final code before committing.

5. Commit the Resolution

```
$ git add app.py
$ git commit -m "Resolve merge
  conflict in app.py"
[main 1a2b3c4] Resolve merge
conflict in app.py
 1 file changed, 1 insertion(+),
 2 deletions(-)
```

- Add the resolved file.
- Commit with a clear message.
- The conflict is now resolved.

Common Causes

- Two engineers changed the same lines.
- Both changed the same function or block of code.
- One changed a line while the other deleted it.
- Large refactoring and feature changes in the same file.
- Long-lived branches with many changes.

Practice This

1. Create a test repository and a file (e.g., app.py).
2. Make different changes to the same line in two branches.
3. Merge the branches and observe the conflict.
4. Resolve the conflict and commit.
5. Use git diff to verify the final result.

Troubleshooting Tips

- Use git merge --abort if you want to cancel the merge.
- Use git log to review the commit history.
- Test your code after resolving the conflict.
- If you are unsure, ask a team member and review the intended changes.

Key Questions to Ask

1. Which parts of the code were changed by each engineer?
2. What is the intended final code?
3. Can the changes be combined?
4. Does the code work after the resolution?
5. Do I need to test anything before committing?

VERIQTA

PAGE 12

The CI Pipeline Failed: Find the First Real Error

ERIQTA

A GitHub Actions workflow displays a red failure after a developer pushes a change.

Learn the difference between dependency installation failures, test failures, build failures, and deployment failures.

Learning Goals

- Understand how a CI pipeline runs and what each job and step does.
- Differentiate between dependency installation failures, test failures, build failures, and deployment failures.
- Learn how to read workflow logs and find the first meaningful error.
- Use exit codes and step output to identify the real cause of the failure.
- Develop a structured approach to troubleshoot failing pipelines.

Key Takeaway

- A red pipeline means one or more steps failed.
- The final summary is not always the real cause.
- The first meaningful error usually appears earlier in the logs.
- Dependency installation failures happen before the build or tests run.
- Test failures mean the code ran but tests failed.
- Build failures occur when the code cannot be compiled or packaged.
- Deployment failures happen when the artefact cannot be deployed to the target environment.

1. View the Failed Workflow

CI Pipeline failed push

Summary

Jobs

- build
- test
- deploy

- Check the workflow overview to see which job failed.
- Expand the failed job to view the steps.
- Note the order of jobs and steps to understand what ran before the failure.

2. Check the Failed Step Logs

Run npm install 10s

```
1 Run npm install
2 npm ERR! code E404
3 npm ERR! 404 Not Found -
  https://registry.npmjs.org/
  example-package - Not found
4 npm ERR! 404
6 npm ERR! 404   'example-package'
  is not in this registry.
```

- Read the logs from top to bottom.
- Find the first meaningful error message.
- Ignore later errors that are a result of the initial failure.
- Look for clear error codes and messages.

3. Understand Common Failure Types

Failure Type	What It Means	Common Clues
Dependency Installation Failure	Required packages cannot be installed.	npm install, pip install, apt-get errors, network issues, missing packages.
Test Failure	The code runs but tests fail.	Test assertion errors, failed tests, non-zero exit code (e.g. exit 1).
Build Failure	The code cannot be compiled or packaged.	Compilation errors, syntax errors, missing dependencies.
Deployment Failure	The artefact cannot be deployed to the target environment.	Authentication errors, permission denied, invalid configuration, service errors.

4. Check Exit Codes

Run pytest 2s

```
1 > pytest
  ========================= test session starts
  FAILED tests/test_app.py::test_login
    assert 200 == 201
  ==================1 failed, 5 passed in 0.45s
  Error: Process completed with exit code 1
```

- Exit code 0 means success.
- A non-zero exit code (e.g. 1) means failure.
- The exit code helps confirm which step actually failed.

5. Find the First Real Error

build 12s

- Set up job 1s
- Checkout code 2s
- Run npm install 10s
- Run build
- Upload artifact

- Always find the earliest failed step in the workflow.
- Fix the first real error, then rerun the pipeline.
- Later steps may fail because earlier steps did not complete.

Practice This

1. Open a failed GitHub Actions workflow.
2. Identify which job and step failed.
3. Read the logs and find the first meaningful error.
4. Determine the type of failure (dependency, test, build, or deployment).
5. Explain the likely cause and suggest a fix.

Troubleshooting Tips

- Read the logs from the beginning.
- Look for the first clear error message.
- Check for network issues if dependencies cannot be installed.
- Verify that required secrets and environment variables are set.
- Rerun the workflow after fixing the issue.
- Use local testing (e.g. run tests or build commands locally) to reproduce the problem.
- Check the workflow file (e.g. .github/workflows/*.yml) for misconfigurations.

Key Questions to Ask

1. What is the first step that failed?
2. What does the error message mean?
3. Is it a dependency, test, build, or deployment failure?
4. What needs to be fixed?
5. Are there any missing secrets or configuration?

VERIQTA

PAGE 13

The Deployment Succeeded, but the App Is Broken

VERIQTA

A deployment tool reports success, but users cannot complete requests.

Learn why deployment completion does not prove application health. Introduce smoke tests, application health checks, and rollback decisions.

Learning Goals

- Understand why a "successful deployment" does not guarantee a healthy application.
- Learn the difference between deployment success and application health.
- Use smoke tests and health checks to verify that the application is actually working.
- Compare the last known working version with the new release.
- Decide when a rollback is safer than continued investigation.

Key Takeaway

- A deployment can succeed, but the application can still be broken.
- Deployment completion only means the code was delivered and started, not that it works correctly.
- Smoke tests quickly check critical functionality after deployment.
- Application health checks verify that the service is healthy and ready to handle real traffic.
- If the new version causes issues, a rollback is often safer and faster than a long investigation.

1. Check the Deployment Status

```
$ kubectl rollout status deployment/app
deployment/app successfully rolled out
$ aws deploy get-deployment
Status: Succeeded
```

- The deployment tool shows the deployment completed successfully.
- This only means the new version was delivered and started.
- It does not confirm that the application is working correctly.

2. Verify Application Health

```
$ curl -i https://example.com/health
HTTP/1.1 500 Internal Server Error
Content-Type: application/json
{"status": "error"}
```

- Check the application's health endpoint.
- Look for expected status codes (e.g., 200 OK).
- A 500 error meams the application is running but not working correctly.
- Compare with the last known working version.

3. Run a Smoke Test

```
$ curl -i https://example.com/api/login
HTTP/1.1 200 OK
Content-Type: application/json
{"message": "success"}
```

- A smoke test checks critical functionality after deployment.
- Test key user actions (login, create, read, etc.).
- If smoke tests fail, the application is not healthy even if the deployment succeeded.

4. Compare the Last Working Version with the New Release

```
# Check current version
$ kubectl get deployment app -o yaml | grep image
  image: myapp:2.0.0
# Check previous version
$ kubectl rollout history deployment/app
  REVISION  CHANGE-CAUSE
  1         myapp:1.9.0  (stable)
  2         myapp:2.0.0  (current, broken)
```

- Identify the last known working version.
- Compare configuration, environment variables, and recent code changes.
- Check what changed between the versions.

5. Decide: Investigate or Rollback?

Situation	Action	Reason
Critical user functionality is broken	Rollback	Restore service quickly and minimize impact.
Minor issue, non-critical	Investigate	Fix may be quick and low risk.
Root cause is unknown and users are affected	Rollback	Safer than prolonged downtime.
You can easily reproduce and fix the issue	Investigate	No need to rollback if fix is fast and safe.

6. Perform a Rollback (Example)

```
# Kubernetes rollback
$ kubectl rollout undo deployment/app
deployment.apps/app rolled back
# Verify the rollback
$ kubectl rollout status deployment/app
deployment/app successfully rolled out
```

- Rollback restores the previous working version.
- Verify the application is healthy again after rollback.
- Investigate the issue in a safe environment before redeploying.

Practice This

1. Check the deployment status for your application.
2. Test the application's health endpoint.
3. Run a smoke test for a key user action.
4. Compare the current version with the last working version.
5. Decide whether to rollback or continue investigating.

Troubleshooting Tips

1. Check application logs for errors.
2. Verify environment variables and configuration.
3. Ensure all dependent services are running.
4. Test from both inside and outside the cluster.
5. If in doubt and users are affected, rollback.

Key Questions to Ask

1. Is the deployment actually serving the new version?
2. Are health checks passing?
3. What errors appear in the application logs?
4. Is the issue in the new code or configuration?
5. Will a rollback resolve the issue quickly and safely?

PAGE 14

Docker Container Exited: Why Did It Stop?

VERIQTA

A container starts successfully but exits moments later.

Learn about the container's main process, exit codes, application crashes, and why restarting a broken container may repeat the failure.

Learning Goals

- Understand what a container is and the role of the main process.
- Learn why a container can exit immediately.
- Understand exit codes and what they mean.
- Use docker ps -a, docker logs, and docker inspect to troubleshoot.
- Know when to fix the application, configuration, or image instead of just restarting the container.

Key Takeaway

- A container runs a main process. When that process exits, the container also stops.
- A container can exit due to application errors, misconfiguration, missing files, or failed dependencies.
- The exit code tells you why the main process stopped.
- Restarting a broken container will usually repeat the same failure if the underlying issue is not fixed.
- Use logs and inspection tools to find the real error before making changes.

1. Check Container Status

```
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND
a1b2c3d4e5f6   myapp:1.0      "python app.py"
STATUS                     PORTS     NAMES
Exited (1) 10 seconds ago            web
```

- docker ps -a shows all containers, including stopped ones.
- The STATUS column shows the exit code and how long ago it stopped.
- An exited container means the main process finished or failed.

2. View the Container Logs

```
$ docker logs a1b2c3d4e5f6
Traceback (most recent call last):
  File "app.py", line 10, in <module>
    import settings
ModuleNotFoundError: No module
named 'settings'
```

- docker logs shows the output from the container's main process.
- Look for error messages, stack traces, or missing dependencies.
- The logs usually reveal the exact reason the container exited.

3. Inspect the Exit Status

```
$ docker inspect a1b2c3d4e5f6 \
  --format '{{.State.ExitCode}}'
1
```

- docker inspect shows detailed information about the container.
- The .State.ExitCode value shows the actual exit code.
- Exit code 0 usually means success.
- A non-zero exit code (e.g. 1, 2, 127) means an error occurred.

4. Common Reasons for Container Exit

Reason	Description	Example
Application error	The main process crashes.	Unhandled exception in the code.
Missing dependency	Required packages or files are not available.	ModuleNotFoundError
Misconfiguration	Environment variables or config files are incorrect.	Cannot connect to database.
Invalid command	The container runs a command that exits immediately.	-docker run image (no long-running process)
Process completes	The main process finishes its job and exits normally.	A script that runs and ends.

5. Understand Common Exit Codes

Exit Code	Meaning
0	Success (process completed).
1	General error in the application.
2	Misuse of shell command.
126	Command found but not executable.
127	Command not found.
137	Process killed (e.g. out of memory).
143	Process terminated (SIGTERM).

6. Fix and Verify

1. Read the logs and identify the real error.
2. Fix the application code, configuration, or dependencies.
3. Rebuild the image if nneded (docker build).
4. Run the container again.
5. Verify it stays running.

```
$ docker run -d --name web myapp:1.0
$ docker ps
CONTAINER ID   IMAGE        STATUS
f9e8d7c6b5a4   myapp:1.0    Up 5 minutes
```

Practice This

1. Run a container that exits immediately.
2. Use docker ps -a to check its status and exit code.
3. Use docker logs to find the error.
4. Use docker inspect to confirm the exit code.
5. Fix the issue and run the container again.

Troubleshooting Tips

- Always check logs first.
- Look for missing files, environment variables, or dependencies.
- Verify the command and entrypoint in the image.
- Ensure the container has a long-running process if it should stay up.
- Do not keep restarting a container without fixing the underlying issue.

Key Questions to Ask

1. What is the container's exit code?
2. What error messages appear in the logs?
3. Is the main process crashing or finishing normally?
4. Are required files, dependencies, or environment variables missing?
5. Do I need to fix the code, configuration, or the container image?

VERIQTA

PAGE 15

The Container Is Running, but Nobody Can Reach It

VERIQTA

Docker reports that the container is running, yet requests to the application fail.

Learn about container networking, published ports, application listening addresses, and the difference between container health and accessibility.

Learning Goals

- Understand how container networking works.
- Learn what published ports are and how they map to the host.
- Understand the difference between a container running and the application being accessible.
- Identify common port-mapping and binding issues.
- Use docker port, docker inspect, and curl to troubleshoot connectivity problems.

Key Takeaway

- A container can be running, but the application may not be reachable.
- The container must publish the correct port to the host.
- The application inside the container must listen on the correct address (e.g. 0.0.0.0, not 127.0.0.1).
- Container health only tells you whether the process is running, not whether the service is accessible from outside.
- Use docker port, docker inspect, and curl to find where the connection is failing.

1. Check the Container Status

```
$ docker ps
CONTAINER ID   IMAGE        STATUS        PORTS
a1b2c3d4e5f6   myapp:1.0    Up 5 minutes
$ docker ps -a
CONTAINER ID   IMAGE        STATUS
a1b2c3d4e5f6   myapp:1.0    Up 5 minutes
```

- The container is running, which means the main process is active.
- Running does not mean the application is reachable from outside.

2. Check Published Ports

```
$ docker port a1b2c3d4e5f6
8080/tcp -> 0.0.0.0:8080
$ docker ps
  PORTS
  0.0.0.0:8080->8080/tcp
```

- docker port shows which container ports are published to the host.
- If there are no published ports, the application will not be reachable from the host.
- Make sure the host port and container port are correct.

3. Inspect the Container Configuration

```
$ docker inspect a1b2c3d4e5f6 | grep -A 5
"Ports"
   "Ports": {
      "8080/tcp": [
         {
            "HostIp": "0.0.0.0",
            "HostPort": "8080"
         }
      ]
   }
```

- docker inspect shows detailed port mappings and network settings.
- Verify the host IP, host port, and container port.
- Check the container's network (bridge, custom network, etc.).

4. Verify the Application Is Listening on the Correct Address

```
# Inside the container
$ docker exec -it a1b2c3d4e5f6 ss -lntp
State     Local Address:Port     Process
LISTEN    0.0.0.0:8080           python
LISTEN    127.0.0.1:8080         python
```

- The application should listen on 0.0.0.0 to accept connections from outside the container.
- If it listens only on 127.0.0.1, it will not be reachable from the host or other containers.

5. Test the Connection with curl

```
# From the host
$ curl -v http://localhost:8080
> GET / HTTP/1.1
> Host: localhost:8080
< HTTP/1.1 200 OK
# If it fails
$ curl -v http://localhost:8080
* connect to 127.0.0.1 port 8080 failed:
  Connection refused
```

- Use curl to verify the application is reachable on the expected host port.
- A connection refused error means nothing is listening on that port.
- A timeout may indicate a firewall or network issue.

6. Common Causes and Fixes

Cause	Symptom	Fix
Port not published	Connection refused	Run with -p host:container (e.g. -p 8080:8080)
App listens on 127.0.0.1	Cannot reach from host	Configure app to listen on 0.0.0.0
Wrong port mapping	Connection refused	Check docker port and update -p mapping
Firewall/host network issue	Timeout	Check firewall rules and host network
App not actually running	Connection refused	Check application logs and configuration

Practice This

1. Run a container with a published port (e.g. -p 8080:8080).
2. Check the port mapping with docker port.
3. Inspect the container with docker inspect.
4. Verify the application is listening on 0.0.0.0 inside the container.
5. Test the service with curl from the host and your browser.
6. Fix any port mapping or binding issue you find.

Troubleshooting Tips

- Always check container logs for application errors.
- Verify the application is listening on 0.0.0.0.
- Confirm the correct port is published to the host.
- Check the container's network and IP address.
- Test from inside and outside the container.
- Look for firewall rules or security groups blocking the port.

Key Questions to Ask

1. Is the container running?
2. Is the correct port published to the host?
3. What address is the application listening on (0.0.0.0 or 127.0.0.1)?
4. Does curl return a response or a connection error?
5. Are there firewall rules blocking the port?
6. What does the container log show?

VERIQTA

PAGE 16

Kubernetes CrashLoopBackOff: Follow the Restart Trail

VERIQTA

A Pod repeatedly starts, crashes, and waits before restarting.

Learn about Pod status, container termination, restart backoff, failed probes, and missing configuration.

Learning Goals

- Understand what CrashLoopBackOff means and why it happens.
- Learn how Pod status, container termination, and restart backoff work.
- Identify whether the restart is caused by an application crash, a failed probe, or missing configuration.
- Use kubectl get pods, kubectl describe pod, and kubectl logs --previous to troubleshoot.
- Develop a structured approach to find and fix the real cause.

Key Takeaway

- CrashLoopBackOff means the container is crashing and Kubernetes is restarting it with an increasing delay (backoff).
- A Pod can be in a running state briefly and still restart if the container exits with an error.
- The cause can be application errors, failed liveness/readiness probes, missing environment variables or configuration, or dependency issues.
- Use `kubectl describe pod` and previous logs to see the exact reason for the restart.
- Fix the root cause instead of just increasing restart limits or delays.

1. Check the Pod Status

```
$ kubectl get pods
NAME                READY  STATUS             RESTARTS  AGE
web-7d9f6c9-cxk2m   0/1    CrashLoopBackOff   5         2m
api-6b7d8f9-plt4z   1/1    Running            0         10m
```

- The STATUS shows CrashLoopBackOff when the container keeps crashing.
- RESTARTS increases each time the container restarts.
- READY shows how many containers are running and ready.

2. Describe the Pod

```
$ kubectl describe pod web-7d9f6c9-cxk2m
  State:          Waiting
  Reason:         CrashLoopBackOff
  Last State:     Terminated
  Reason:         Error
  Exit Code:      1
  Started:        Fri, 24 Oct 2026 10:01:12
  Finished:       Fri, 24 Oct 2026 10:01:15
```

- Shows the current and last container state.
- The Reason and Exit Code indicate why the container stopped.
- Look for events at the bottom for probe failures, image pull errors, or configuration issues.

3. Check the Previous Logs

```
$ kubectl logs web-7d9f6c9-cxk2m
  --previous
Error: Could not connect to database
Caused by: dial tcp 10.96.0.10:5432:
connection refused
exit status 1
```

- --previous shows logs from the last terminated container.
- Look for application errors, missing files, failed connections, or configuration issues.
- The error message usually explains why the container exited.

4. Common Causes of CrashLoopBackOff

Cause	Example
Application crash	App exits with an error (e.g. exit code 1).
Failed liveness probe	Probe cannot reach the app, so Kubernetes restarts the container.
Failed readiness probe	App is not ready and fails the probe.
Missing configuration	Missing environment variables or config files.
Dependency not available	Cannot connect to database, API, or other service.
Invalid command or args	Wrong startup command or arguments.
Resource issues	Out of memory or insufficient resources.

5. Check for Probe-Related Restarts

```
$ kubectl describe pod web-7d9f6c9-cxk2m
  | grep -A 5 "Liveness"
Liveness:         http-get http://:8080/health
    Initial delay:      10s
    Period:             10s
    Timeout:            1s
    Failure threshold:  3
```

- If the liveness probe fails, Kubernetes restarts the container.
- Check that the probe endpoint, port, and path are correct.
- Ensure the application is ready before the probe starts.

6. Fix and Verify

1. Identify the root cause from logs, describe output, and events.
2. Fix the application code, configuration, or dependencies.
3. Deploy the updated image or configuration.
4. Monitor the pod until it stays in the Running state.
5. Verify the application is healthy and passing probes.

Practice This

1. Check the status of a crashing Pod.
2. Describe the Pod and find the termination reason.
3. View the previous logs and identify the error.
4. Determine whether the cause is an application crash, probe failure, or missing configuration.
5. Fix the issue and confirm the Pod stays Running.

Troubleshooting Tips

- Always check both current and previous logs.
- Look at the exit code and error message.
- Verify environment variables and configuration files are set correctly.
- Check application and probe endpoints locally.
- Review events with kubectl describe pod.
- Look for dependency or network issues.

Key Questions to Ask

1. What is the Pod's current status?
2. What is the container's exit code and error message?
3. Are the liveness or readiness probes failing?
4. Are any configuration or environment variables missing?
5. Can the application connect to its dependencies?
6. What changed recently?

PAGE 17

ImagePullBackOff: Why Kubernetes Cannot Start the Image

VERIQTA

A Pod remains pending container startup because Kubernetes cannot retrieve its image.

Explain image references, registry authentication, missing tags, and image pull errors.

Learning Goals

- Understand what ImagePullBackOff means and why it happens.
- Learn how Kuberentes pulls images from a registry.
- Understand image references, tags, and registry authentication.
- Identify common image pull errors and their causes.
- Use kubectl describe pod and events to investigate.
- Verify the image name, tag, and registry access.

Key Takeaway

- ImagePullBackOff means Kubernetes cannot download the container image.
- This often happens due to a wrong image name or tag, a missing image, or authentication/permission issues.
- The Pod stays in a waiting state and repeatedly tries to pull the image with increasing delays.
- Pod events show the exact reason for the failure.
- Fix the image reference, tag, or registry access issue, then Kubernetes will be able to start the container.

1. Check the Pod Status

```
$ kubectl get pods
NAME              READY  STATUS             RESTARTS  AGE
web-7d9f6c9-ckx2m 0/1    ImagePullBackOff   0         5m
```

- The STATUS shows ImagePullBackOff when Kubernetes cannot pull the container image.
- READY shows 0/1 because the container has not started.
- RESTARTS usually stays at 0 since the container never starts.

2. Describe the Pod and Check Events

```
$ kubectl describe pod web-7d9f6c9-ckx2m
Events:
  Type     Reason     Age   Message
  Normal   Scheduled  5m    Successfully assigned to node
  Warning  Failed     4m    Failed to pull image "myapp:1.0"
  Warning  Failed     4m    rpc error: code = NotFound desc = failed to resolve reference
  Warning  BackOff    2m    Back-off pulling image "myapp:1.0"
```

- The Events section shows the exact reason why the image pull failed.
- Look for the error message, such as NotFound (image does not exist) or access denied (authentication issue).

3. Verify the Image Reference

```
$ kubectl get pod web-7d9f6c9-ckx2m
  -o yaml | grep image:
    image: myapp:1.0
```

- Check the image name and tag in the Pod specification.
- Make sure the image name is correct (e.g., myapp) and the tag exists (e.g., 1.0, latest, v1.2.3).
- Typos or missing tags will cause image pull errors.
- You can also check the image reference in your Deployment or Pod manifest.

4. Common Image Pull Errors and Causes

Error Message	Meaning	Common Cause
NotFound	Image does not exist in the registry.	Wrong image name or tag.
ErrImagePull	Failed to pull the image.	Network issue, registry down, or invalid image.
ImagePullBackOff	Repeated failures to pull the image.	Kubernetes is retrying after failed pulls.
unauthorized: access denied	Authentication failed.	Missing or incorrect registry credentials.
x509: certificate signed by unknown authority	TLS/SSL certificate issue.	Self-signed certificate or untrusted registry.
denied: requested access to the resource is denied	Permission denied.	Insufficient role or access rights.

5. Check Registry Access (Example)

```
# For Docker Hub(public image)
$ docker pull nginx:1.25
1.25: Pulling from library/nginx
Digest: sha256:....
Status: Downloaded newer image for nginx:1.25
```

```
# For private registry (requires login)
$ docker login myregistry.com
Username: your-username
Password: **********
Login Succeeded
$ docker pull myregistry.com/myapp:1.0
```

- Try pulling the image manually to verify it exists and you have access.
- For private registries, make sure you are logged in and using the correct credentials.
- If the image cannot be pulled, fix the image name, tag, or registry authentication.

6. Fix and Verify

1. Correct the image name or tag if it is wrong.
2. If using a private registry, create or update the imagePullSecret with the correct credentials.
3. Ensure the image exists in the registry.
4. Apply the updated manifest or redeploy the application.
5. Check the Pod status again to confirm the image pulls successfully and the container starts.

Practice This

1. Check the status of a Pod with ImagePullBackOff.
2. Describe the Pod and find the image pull error.
3. Verify the image name and tag in the manifest.
4. Test pulling the image from the registry.
5. Fix the issue and confirm the Pod starts successfully.

Troubleshooting Tips

- Always check the Pod events for the exact error message.
- Verify the image name and tag are correct.
- Test pulling the image manually from the registry.
- For private registries, ensure the imagePullSecret is configured correctly.
- Check network connectivity to the registry.
- If using a custom registry, verify TLS certificates and access rights.

Key Questions to Ask

1. What is the exact error message in the Pod events?
2. Does the image name and tag exist in the registry?
3. Do you have the correct registry credentials?
4. Is the registry reachable from the cluster?
5. Are there any network, TLS, or permission issues?
6. What changed recently?

PAGE 18

Kubernetes Service Exists, but Traffic Still Fails

VERIQTA

A Deployment has running Pods and a Service, but requests cannot reach the application.

Explain Service selectors, Pod labels, EndpointSlices, target ports, and readiness.

Learning Goals

- Understand how a Kubernetes Service routes traffic to Pods.
- Learn how Service selectors match Pod labels.
- Understand EndpointSlices and how they show the actual backend Pods.
- Learn the difference between port, targetPort, and containerPort.
- Identify common causes of traffic failures even when Pods are running.
- Use kubectl commands to trace traffic from the Service to the intended Pods.

Key Takeaway

- A Service selects Pods using labels. If labels do not match, no traffic is sent to the Pods.
- EndpointSlices show which Pods are actually receiving traffic from the Service.
- The Service port is what clients use, and the targetPort is the port on the container.
- Pods must be ready to receive traffic. Not Ready Pods are not added to EndpointSlices.
- A Service can exist and have a ClusterIP or external access, but traffic will still fail if there are no matching, ready endpoints.

1. Check the Service Details

```
$ kubectl get svc
NAME       TYPE        CLUSTER-IP     PORT(S)
web-svc    ClusterIP   10.96.12.34    80/TCP

$ kubectl describe svc web-svc
Selector:     app=web
Type:         ClusterIP
IP:           10.96.12.34
Port:         80/TCP
TargetPort:   8080/TCP
```

- Verify the Service type, cluster IP, port, targetPort, and selector.
- The selector must match the labels on your Pods.
- The targetPort must match the actual container port.

2. Check the Pod Labels

```
$ kubectl get pods --show-labels
NAME        READY   STATUS    LABELS
web-7d9c6   1/1     Running   app=web
web-7d9c7   1/1     Running   app=web
api-5f8d4   1/1     Running   app=api
```

- Check that the Pod labels match the Service selector.
- If labels do not match, the Service will not send traffic to the Pods.
- Make sure the Pods are in Ready state (1/1). Not Ready Pods will not receive traffic.

3. Check the EndpointSlices

```
$ kubectl get endpointslices
-l kubernetes.io/service-name=web-svc
NAME            ADDRESSES     PORTS
web-svc-abc12   10.244.1.5    8080
web-svc-def34   10.244.2.7    8080
```

- EndpointSlices show which Pod IPs are receiving traffic from the Service.
- If no addresses are listed, the selector does not match any ready Pods.
- Verify that the port shown matches the targetPort.
- If the list is empty, check labels, readiness probes, or Pod status.

4. Test Connectivity

```
# From inside the cluster (using a test pod)
$ kubectl run -it --rm test-pod --image=busybox
 /bin/sh
$ wget -qO-. http://web-svc
# Or test a specific Pod directly
$ kubectl port-forward pod/web-7d9c6 8080:8080
$ curl -i http://localhost:8080
```

- Test the Service DNS name from inside the cluster.
- If the Service is internal, test from another Pod.
- You can also port-forward to a Pod to verify the application is responding.

5. Common Causes and Fixes

Cause	Symptom	Fix
Selector does not match Pod labels	No endpoints in EndpointSlices	Update Service selector or Pod labels to match.
Pods not ready	Endpoints missing or fewer than expected	Check readiness probes, fix application issues.
Wrong targetPort	Connection refused	Set targetPort to the correct container port.
Pod listening on 127.0.0.1	Cannot reach from other Pods	Ensure the application listens on 0.0.0.0.
Network policy blocking traffic	Timeout or connection refused	Check and update NetworkPolicy rules.
Service type misunderstood	Accessible only inside cluster	Use NodePort or LoadBalancer if external access is required.

Practice This

1. Check the Service details and identify the selector and targetPort.
2. Check the Pod labels and confirm they match the selector.
3. View the EndpointSlices and verify that the expected Pods are listed.
4. Test the Service from inside the cluster using a temporary Pod.
5. Fix a misconfigured selector or targetPort and verify that traffic reaches the application.

Troubleshooting Tips

- Always check the Service selector, Pod labels, and EndpointSlices.
- Verify that the Pods are Ready and listening on the correct port.
- Use port-forwarding to test the application directly.
- Check NetworkPolicies if traffic is blocked.
- Look at application logs if connections are refused.
- Test from both inside and outside the cluster when using NodePort or LoadBalancer.

Key Questions to Ask

1. Do the Service selector and Pod labels match?
2. Are there any endpoints in the EndpointSlices?
3. Are the Pods in Ready state?
4. Is the targetPort correct?
5. Is the application listening on 0.0.0.0?
6. Could a network policy be blocking the traffic?

PAGE 19

Cloud Server Unreachable: Find the Broken Layer

VERIQTA

An application running on an AWS EC2 instance is no longer accessible from outside.

Explain instance state, public IP addressing, security groups, network ACLs, host firewalls, and application listeners.

aws

Learning Goals

- Understand the flow of a request from the client to an EC2 instance.
- Learn each network and security layer that can block access.
- Identify common reasons an instance becomes unreachable.
- Use the right tools and commands to check each layer.
- Troubleshoot safely without opening all inbound ports or weakening security.

Key Takeaway

- A connection can fail at multiple layers: your local network, the internet, AWS networking, security groups, network ACLs, the instance, or the application.
- The instance can be running but still unreachable.
- Security groups control traffic to the instance.
- Network ACLs control traffic at the subnet level.
- The application must be listening on the correct IP address and port.
- Check each layer systematically instead of guessing or opening all ports.

1. Check the EC2 Instance State

```
$ aws ec2 describe-instances \
  --instance-ids i-0123456789abcdef0 \
  --query 'Reservations[].Instances[].
{State:State.Name,PublicIp:PublicIpAddress,
PrivateIp:PrivateIpAddress}'
[{
   "State": "running",
   "PublicIp": "54.12.34.56",
   "PrivateIp": "10.0.1.10"
}]
```

- Verify the instance is in the running state.
- Confirm it has a public IP if you expect internet access.
- Note the private IP for internal testing.

2. Test from Your Local Machine

```
$ ping 54.12.34.56
$ curl -v http://54.12.34.56:80
*   Trying 54.12.34.56 :80...
*  connect() timed out
curl: (28) Connection timed out
```

- Check basic network connectivity.
- Test the specific application port (e.g., 22 for SSH, 80 for HTTP, 443 for HTTPS).
- A timeout usually means a network or firewall issue. A connection refused means the instance is reachable but the application is not listening.

3. Check the Security Group

```
$ aws ec2 describe-security-groups \
  --group-ids sg-0123456789abcdef0
```

Inbound Rules:

Type	Protocal	Port Range	Source
SSH	TCP	22	0.0.0.0/0
HTTP	TCP	80	0.0.0.0/0
HTTPS	TCP	443	0.0.0.0/0

- Verify that the required port is allowed.
- Check the source (CIDR) and protocol.
- Only allow trusted IP ranges (avoid 0.0.0.0/0 for sensitive services).
- Ensure the security group is attached to the correct instance.

4. Check the Network ACL (Subnet Level)

```
$ aws ec2 describe-network-acls \
  --network-acl-ids acl-0123456789abcdef0
```

Inbound Rules:

Rule #	Protocol	Port Range	Source	Action
100	TCP	80	0.0.0.0/0	allow
110	TCP	443	0.0.0.0/0	allow

- Verify the subnet's network ACL allows the required inbound and outbound traffic.
- Check both inbound and outbound rules (ACLs are stateless).
- Ensure the instance is in the correct subnet.

5. Check the Host Firewall

```
# On the EC2 instance (Linux)
$ sudo ufw status
Status: active
```

To	Action	From
22/tcp	ALLOW	Anywhere
80/tcp	ALLOW	Anywhere
443/tcp	ALLOW	Anywhere

- Check the OS firewall (ufw, firewalld, or iptables).
- Ensure the required port is allowed.
- Verify the firewall is not blocking traffic from your IP.

6. Check the Application Listener

```
# On the EC2 instance
$ sudo ss -tulnp | grep :80
Netid   State    Local Address:Port
tcp     LISTEN   0.0.0.0:80
or
$ sudo netstat -tulnp | grep :80
```

- Verify the application is running.
- Check that it is listening on the correct port and address (0.0.0.0 or the instance's private IP).
- If it is only listening on 127.0.0.1, it will not be accessible from outside.

Practice This

1. Check the EC2 instance state and public IP.
2. Test connectivity from your local machine using ping and curl.
3. Review the security group rules for the required port.
4. Verify the network ACL rules for the subnet.
5. Check the host firewall and application listener.
6. Fix the issue and confirm the application is accessible.

Troubleshooting Tips

- Test each layer one at a time.
- Use VPC Flow Logs or CloudWatch for deeper analysis if needed.
- Do not open all inbound ports. Allow only required ports and trusted IP ranges.
- If using a load balancer, also check the target group and health checks.
- Verify DNS records if using a domain name.

Key Questions to Ask

1. Is the instance running?
2. Does it have a public IP?
3. Is the security group allowing the traffic?
4. Are the network ACL rules correct?
5. Is the host firewall blocking the port?
6. Is the application listening on the correct address and port?
7. Are you using a load balancer or DNS?

PAGE 20

The Production Incident: Diagnose, Recover, and Prove It

VERIQTA

A new application release causes intermittent HTTP 502 errors.

One backend instance is restarting, and the monitoring dashboard shows an increase in failed requests. **This is the final integrated troubleshooting challenge.**



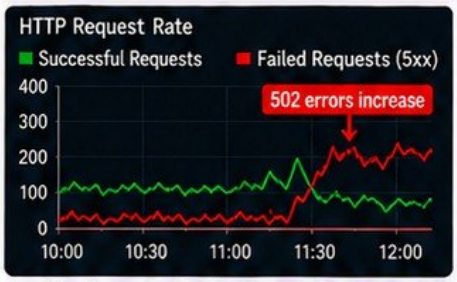
Learning Goals

- Understand how to approach a real production incident.
- Identify the impact on users and services.
- Correlate deployment timing with logs and monitoring data.
- Investigate the failing instance and find the root cause.
- Choose a safe recovery action.
- Verify service restoration and document the incident.
- Create a complete incident record (evidence, root cause, action, verification, prevention).

Key Takeaway

- Intermittent HTTP 502 usually means the load balancer cannot get a valid response from the backend.
- A single unhealthy instance can cause some requests to fail even if other instances are healthy.
- Always check deployment time, instance health, application logs, and monitoring metrics together.
- The root cause can be application errors, failed health checks, missing configuration, resource issues, or a bad release.
- Recover by fixing the issue, restarting the instance, or rolling back to the previous stable version.
- Verify the service is fully working before closing the incident.
- Document what happened and how to prevent it next time.

1. Confirm the Impact



- Check monitoring dashboard for 5xx errors.
- Identify when the failures started.
- See which services and users are affected.
- Check if the issue is global or limited to one instance or region.

2. Correlate Deployment and Logs

```
$ kubectl get pods
NAME          READY  STATUS            RESTARTS  AGE
web-7d9c6c-1  1/1    Running           0         2h
web-7d9c6c-2  0/1    CrashLoopBackOff  5         10m
web-7d9c6c-3  1/1    Running           0         2h
```

```
$ kubectl logs web-7d9c6c-2 --previous
Error: Could not connect to database
Caused by: timeout after 30s
 at app.js:45
```

- Compare the time of the release with the start of the errors.
- Check pod/instance status and restart count.
- Review application logs for the exact error.
- Look for configuration changes or dependency issues in the new release.

3. Investigate the Failing Instance

```
$ kubectl describe pod web-7d9c6c-2
  State:        Waiting
  Reason:       CrashLoopBackOff
  Last State:   Terminated
  Reason:       Error
  Exit Code:    1
```

```
$ kubectl logs web-7d9c6c-2
Error: Database connection failed
Retrying in 5 seconds...
Error: Connection timeout
```

- Find the exact reason the container is failing.
- Check environment variables and secrets.
- Verify the application can reach its dependencies (e.g., database, API).
- Check resource usage (CPU, memory, disk).
- Look for failed health or readiness probes.

4. Recover the Service

```
# Option 1: Roll back to previous version
$ kubectl rollout undo deployment/web
# Option 2: Restart the failing instance
$ kubectl delete pod web-7d9c6c-2
# Option 3: Fix configuration and redeploy
$ kubectl apply -f deployment.yaml
```

- Choose the safest recovery action.
- Roll back if the new release is the cause.
- Restart the instance after fixing the issue.
- Redeploy with the correct configuration.
- Monitor to confirm the service is stable.

5. Verify Service Restoration

```
$ kubectl get pods
NAME          READY  STATUS   RESTARTS
web-7d9c6c-1  1/1    Running  0
web-7d9c6c-2  1/1    Running  0
web-7d9c6c-3  1/1    Running  0
```

```
$ curl -I https://yourapp.com
HTTP/1.1 200 OK
```

- Confirm all instances are healthy.
- Check the monitoring dashboard for normal error rates.
- Test the application endpoints (e.g., curl).
- Verify users can complete their requests.
- Keep monitoring for a period after recovery.

6. Incident Record (Example)

Section	Details (Example)
Incident Title	Intermittent HTTP 502 after new release
Start Time	2026-10-24 11:05 UTC
Impact	Some users received 502 errors
Root Cause	New release had a database connection timeout
Corrective Action	Rolled back to previous version
Verification	All instances healthy, 5xx errors returned to normal
Prevention	Add startup checks, better error handling, and pre-release testing

Practice This

1. Examine the monitoring dashboard and identify the impact.
2. Find the failing instance and check its logs.
3. Identify the root cause from the logs and configuration.
4. Recover the service using a safe action.
5. Verify the service is restored.
6. Complete a short incident record.

Troubleshooting Tips

- Always check deployment time and changes.
- Look at logs from the failing instance.
- Check dependencies (database, API, external services).
- Verify environment variables and secrets.
- Monitor CPU, memory, and disk usage.
- Check health and readiness probe configuration.
- Use rollback if you need a quick and safe recovery.

Key Questions to Ask

1. When did the 502 errors start?
2. Which users or services are affected?
3. Which instance is failing and why?
4. What changed in the recent release?
5. Can the application reach its dependencies?
6. What is the safest recovery action?
7. How can we prevent this from happening again?